claude-windows / 75d3d121-f240-4293-afa5-b436a379ad31

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-khelsutra-guru-badminton-highlight-indexer\75d3d121-f240-4293-afa5-b436a379ad31\subagents\workflows\wf_5f13fc79-bad\agent-ab5995df7f12953f4.jsonl`
- SHA-256: `8d1cfd8fe680531f5131ba9f17b15f24b353fce39f51d8fc7f4e1ad40e3bea11`
- Source modified: `2026-07-08T06:18:05+00:00`
- Imported at: `2026-07-08T15:59:28+00:00`
- project: `wf_5f13fc79-bad`
- session_id: `75d3d121-f240-4293-afa5-b436a379ad31`

Transcript

1. user (2026-07-08T06:14:14.437Z)

CONTEXT: This is GitHub issue #521 (Khelsutra/badminton-highlight-indexer), MERGED as PR #525. It makes pipeline phase?machine placement config-driven and dispatches the reel compile/stitch to
an L4 Cloud Run worker (NVENC), mirroring the existing detection dispatch.

The full unified diff is at: undefined (read it first).
The full post-change source is under: undefined/ (read whole files there for context ? e.g. undefined/backend/orchestration.py, undefined/backend/worker/__main__.py, undefined/backend/compute/cloud_run.py, undefined/backend/config/placement.py, undefined/backend/config/models.py, undefined/backend/pipeline/stitcher/compiler.py).

Key design invariants that MUST hold (a violation is a real bug):
- DEFAULT-OFF: with no phase_placement config and compute_target unset, behaviour is byte-identical
  to before #521 (validation+compile on control-plane, detect in-process).
- The cloud compile path must return the SAME {status, filepath, download_url, video_id} shape the
  SPA already polls for; a stitch failure must be a clean failed-job, never a crash.
- The worker 'compile' subcommand reads a bucket-staged spec (resolved time-pairs + stitching config
  + source uri) and needs NO control-plane DB. It reuses orchestration._stitch_reel (shared core).
- Encoder: all clips in one reel must share ONE codec so the concat stays -c copy. NVENC is probed
  once and falls back to libx264 if unavailable (CI/non-GPU) ? the fallback must be all-or-nothing,
  never per-clip (which would break the concat).
- The config guard rejects validation=cloud_run_l4 without a cloud segment (nobody-validates footgun).
- Marker prefixes: compile uses _compile/, detect uses _dispatch/ ? must not collide.

Only report DEFECTS in the changed code (logic bugs, broken invariants, edge cases that produce a wrong result / crash / silent data loss, or missing test coverage of a real failure mode). Do NOT report style, naming, or speculative concerns. Cite exact file:line from undefined.

A reviewer claims this DEFECT in the merged #521 code. Try HARD to REFUTE it by reading the actual code at undefined. Default to is_real=false if you cannot concretely confirm the failure. Skeptic #2.

CLAIM (high/test-coverage) backend/orchestration.py:1263
summary: No test exercises the L4 worker's real _stitch_reel mirror-upload; a swallowed mirror failure there reports success (rc 0) but the reel is lost, so the SPA gets a 404 for a job it was told succeeded.
failure scenario: On the L4, a compile job runs cmd_compile -> orchestration._stitch_reel. The ffmpeg stitch succeeds locally, then `storage.put(out_path, gs://.../highlights/<vid>/<name>)` fails (transient GCS 5xx, throttling, or a creds/permission hiccup). The `except Exception` at orchestration.py:1263 swallows it (logs a warning) and _stitch_reel returns `status="success"` with the intended `reel_uri` (line 1270-1276). cmd_compile (worker/__main__.py:513-514) sets rc=0 purely from that status and writes a SUCCESS done-marker; _dispatch_compile_remote returns {status: success, download_url}. But the reel was never uploaded and the worker's local file is discarded when the Job exits, so /api/highlights -> signed_reel_url -> storage.exists() is False, the control-plane has no local fallback (the stitch ran on the worker), and the download 404s forever. Every #521 test hides this: test_worker.py's cmd_compile tests monkeypatch orchestration._stitch_reel, and test_compile_dispatch.py / test_compute_backend.py use a fake CloudRunJobClient that writes the marker directly, so the real worker mirror path (and its swallow-on-failure) is never run.

2. assistant (2026-07-08T06:14:24.891Z)

I'll investigate this claim by reading the actual code. Let me start with the key files.

3. user (2026-07-08T06:14:26.256Z)

```
1180    def signed_reel_url(cfg, video_id: str, name: str) -> Optional[str]:
1181        """A short-lived signed URL for a reel mirrored to the output bucket, so
/api/highlights can
1182        307-redirect off the app's response path (bypassing Cloud Run's ~32 MiB cap + the
ephemeral local
1183        /tmp; #463). None ? no bucket copy ? serve the local file. Uses the active storage
config for BOTH
1184        the existence check and the signing (extracted here to keep backend/main.py lean ?
P23/P24)."""
1185        reel_uri = reel_object_uri(cfg, video_id, name)
1186        if not reel_uri:
1187            return None
1188        settings = _storage_settings(cfg)
1189        if storage.exists(reel_uri, config=settings):
1190            return storage.signed_url(reel_uri, config=settings, method="GET")
1191        return None
1192
1193
1194    def _compile_wait_message(elapsed_sec: float) -> str:
1195        """The live job.progress line while an L4 compile/stitch is in flight (#521; mirrors
1196        ``_wait_progress_message``). Minute-granular; under a minute reads as cold-start."""
1197        minutes = int(elapsed_sec) // 60
1198        if minutes < 1:
1199            return "stitching on cloud GPU (NVENC) ? starting up"
1200        return f"stitching on cloud GPU (NVENC) ? {minutes} min elapsed"
1201
1202
1203    def _stitch_reel(
1204        cfg,
1205        stitching,
1206        video_id: str,
1207        source_ref: str,
1208        time_pairs: List[Tuple[float, float]],
1209        out_name: str,
1210        locale: Optional[str],
1211        notify,
1212    ) -> Dict[str, Any]:
1213        """Stitch + mirror-upload core, shared by the control-plane compile handler AND the
L4 worker
1214        (#521). ``time_pairs`` are already RESOLVED (the caller filtered via
``_resolve_compile``), so this
```

```
       needs NO control-plane DB ? the worker reads them from the staged spec. Builds the VideoCompiler
       with ``stitching.encoder`` (NVENC on the L4, libx264 on the CP), fetches the source, runs the
       trim/watermark/concat, mirrors the reel to ``<output_root>/highlights/<video_id>/<name>``, and
       returns the ``{status, filepath, download_url, video_id, reel_uri}`` the SPA polls for. Returns a
       ``status='failed'`` result on a stitch error (never raises for an expected ffmpeg failure)."""
       import backend.main as _main  # call-time seam: VideoCompiler stays patchable on backend.main

   <<<<<<< Updated upstream
   =======
       notify = progress or (lambda stage: None)
       params = json.loads(job.get("params") or "{}")
       video_id = job.get("video_id") or ""
       segment_ids = params.get("segment_ids") or []
       locale = params.get("locale")
       rendition = str(params.get("rendition") or "original")
       if rendition not in {"original", "sd"}:
           return {
               "status": "failed",
               "message": f"Unsupported compile rendition: {rendition}",
               "video_id": video_id,
           }

       notify("resolving")
       try:
           video, kept, out_name = _resolve_compile(
               db, cfg, video_id, segment_ids, params.get("output_filename", "")
           )
       except _CompileError as e:
           return {"status": "failed", "message": e.detail, "video_id": video_id}

       stitching = getattr(cfg, "stitching", None) or StitchingConfig()
   >>>>>>> Stashed changes
       output_dir = (
           getattr(getattr(cfg, "output", None), "output_dir", "output") or "output"
       )
       # Localize the branding watermark to the UI locale the SPA rendered in (recorded on the job);
       # unknown/absent locale keeps the configured default unchanged.
       localized_watermark = localize_watermark(stitching.watermark, locale)
       encode_profile = _rendition_encode_profile(stitching, rendition)
       compiler = _main.VideoCompiler(
           output_dir,
           begin_padding=stitching.begin_padding,
           end_padding=stitching.end_padding,
           watermark=localized_watermark,
   <<<<<<< Updated upstream
           encoder=getattr(stitching, "encoder", "libx264"),
   =======
           encode_profile=encode_profile,
   >>>>>>> Stashed changes
       )

       notify("stitching")
       logger.info(
           "[compile] stitching video_id=%s rendition=%s output=%s",
           video_id,
           rendition,
           out_name,
       )
```

```
1273        local_src = ""
1274        try:
1275            # The stored `filepath` is the SOURCE ref (gs://, s3://, gdrive://, or a local
path). Resolve to
1276            # a LOCAL file the stitcher can read ? identity for a local path, fetch-to-
scratch for a URI (the
1277            # same seam the process path uses); without it, compiling a bucket-ingested
video can't open gs://.
1278            local_src = storage.acquire_local_input(
1279                source_ref, getattr(cfg, "storage", None)
1280            )
1281            # The ffmpeg stitch is CPU-heavy ? serialize it so a raised drain worker count
(for remote
1282            # dispatch) doesn't run multiple compiles at once on the box (see
_LOCAL_COMPUTE_LANE).
1283            with _LOCAL_COMPUTE_LANE:
1284                out_path = compiler.compile_highlights(local_src, time_pairs, out_name)
1285        except Exception as e:  # noqa: BLE001 ? a stitch/ffmpeg failure is a clean job
FAILURE, not a crash
1286            logger.error(
1287                "[compile] stitching failed for video_id=%s: %s", video_id, e, exc_info=True
1288            )
1289            return {
1290                "status": "failed",
1291                "message": f"Highlight stitching failed: {e}",
1292                "video_id": video_id,
1293            }
1294        finally:
1295            if local_src:
1296                storage.cleanup_acquired_local_input(
1297                    source_ref, local_src, getattr(cfg, "storage", None)
1298                )
1299
1300        # #463: mirror the reel to the output bucket so /api/highlights can 307 to a signed
URL ? Cloud Run
1301        # caps a buffered response at ~32 MiB (a real reel is bigger) and the local /tmp
copy dies when the
1302        # instance recycles. A mirror-upload failure must NOT fail an otherwise-good stitch
(the endpoint
1303        # still falls back to the local file on the same instance). On the L4 worker the
mirror is how the
1304        # reel reaches the bucket at all (the worker's local file is discarded when the Job
exits).
1305        reel_uri = reel_object_uri(cfg, video_id, out_name)
1306        if reel_uri:
1307            try:
1308                notify("uploading reel")
1309                # Thread the active storage settings so the configured backend (S3
endpoint/region, GCS
1310                # project/client) is used, not a fresh default.
1311                storage.put(out_path, reel_uri, config=_storage_settings(cfg))
1312            except Exception as e:  # noqa: BLE001 ? best-effort mirror; local stitch
already succeeded
1313                logger.warning("[compile] could not mirror reel to %s: %s", reel_uri, e)
1314
1315        # `filepath` is server-local (not browser-reachable); also return the download URL
so the SPA needn't
1316        # hand-build it (P6 / PER_RALLY_SELECTION.md §4.3).
1317        download_url = f"/api/highlights/{quote(video_id)}/{quote(out_name)}"
1318        notify("finished")
1319        return {
```

4. user (2026-07-08T06:14:26.612Z)

backend\worker__main__.py

5. user (2026-07-08T06:14:27.058Z)

```
tests\test_worker.py
```

6. user (2026-07-08T06:14:27.259Z)

```
tests\test_compile_dispatch.py
```

7. assistant (2026-07-08T06:14:56.206Z)

I notice the working copy of `orchestration.py` has git merge-conflict markers (a stash conflict). I need the clean merged version. Let me check git state and read the committed version.

8. user (2026-07-08T06:14:58.986Z)

```
On branch codex/issue-523-sd-reel-plan
Your branch is up to date with 'origin/master'.

Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        modified:   README.md
        modified:   SESSION_HANDOFF.md
        modified:   backend/config/models.py
        modified:   backend/infrastructure/database_jobs.py
        modified:   backend/main.py
        modified:   config.json
        modified:   tests/test_api_endpoints.py
        modified:   tests/test_async_jobs.py
        modified:   tests/test_compiler.py
        modified:   tests/test_watermark.py

Unmerged paths:
  (use "git restore --staged <file>..." to unstage)
  (use "git add <file>..." to mark resolution)
        both modified:   backend/orchestration.py
        both modified:   backend/pipeline/stitcher/compiler.py

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        score_golden_gate.py
        score_local.py

---LOG---
dac6736 perf(serving): run compile/stitch on the L4 worker; config-driven phase?machine
placement (#521) (#525)
bf97700 docs(runlogs): record live SPA CUJ closing the 2 owner-gated validation-latency cells
(§9) (#522)
699d85b feat(validation): cache validation verdict by video_id to skip re-validation on reupload
(#518)
71c88ef docs(runlogs): validation-latency closure report for #512 + #513 (cloud-serving) (#519)
115b83d docs(runlogs): record live smoke of the crash-before-marker fast-fail (#515) (#517)
ef41473 fix(cloud-run): fail fast when a worker execution terminates without a done-marker
(#515)
f5c9dd9 chore(ui): re-vendor SPA bundle with the hosted compute-picker cloud default (#516)
ec2d080 fix(serving): fail fast when compute=local forces weightless in-process detection (#511)
---STASH---
stash@{0}: On codex/issue-523-sd-reel-plan: sd-reel-engine-wip
```

9. assistant (2026-07-08T06:15:05.473Z)

The working tree has a stash-pop conflict on top of `dac6736` (PR #525 = merged #521). Let me read the clean committed versions from HEAD.

10. user (2026-07-08T06:15:08.187Z)

```
def _compile_wait_message(elapsed_sec: float) -> str:
    """The live job.progress line while an L4 compile/stitch is in flight (#521; mirrors
    ``_wait_progress_message``). Minute-granular; under a minute reads as cold-start."""
    minutes = int(elapsed_sec) // 60
    if minutes < 1:
        return "stitching on cloud GPU (NVENC) ? starting up"
    return f"stitching on cloud GPU (NVENC) ? {minutes} min elapsed"


def _stitch_reel(
    cfg,
    stitching,
    video_id: str,
    source_ref: str,
    time_pairs: List[Tuple[float, float]],
    out_name: str,
    locale: Optional[str],
    notify,
) -> Dict[str, Any]:
    """Stitch + mirror-upload core, shared by the control-plane compile handler AND the L4
worker
    (#521). ``time_pairs`` are already RESOLVED (the caller filtered via ``_resolve_compile``),
so this
    needs NO control-plane DB ? the worker reads them from the staged spec. Builds the
VideoCompiler
    with ``stitching.encoder`` (NVENC on the L4, libx264 on the CP), fetches the source, runs
the
    trim/watermark/concat, mirrors the reel to ``<output_root>/highlights/<video_id>/<name>``,
and
    returns the ``{status, filepath, download_url, video_id, reel_uri}`` the SPA polls for.
Returns a
    ``status='failed'`` result on a stitch error (never raises for an expected ffmpeg
failure)."""
    import backend.main as _main  # call-time seam: VideoCompiler stays patchable on
backend.main

    output_dir = (
        getattr(getattr(cfg, "output", None), "output_dir", "output") or "output"
    )
    # Localize the branding watermark to the UI locale the SPA rendered in (recorded on the
job);
    # unknown/absent locale keeps the configured default unchanged.
    localized_watermark = localize_watermark(stitching.watermark, locale)
    compiler = _main.VideoCompiler(
        output_dir,
        begin_padding=stitching.begin_padding,
        end_padding=stitching.end_padding,
        watermark=localized_watermark,
        encoder=getattr(stitching, "encoder", "libx264"),
    )

    notify("stitching")
    local_src = ""
    try:
        # The stored `filepath` is the SOURCE ref (gs://, s3://, gdrive://, or a local path).
Resolve to
        # a LOCAL file the stitcher can read ? identity for a local path, fetch-to-scratch for a
URI (the
        # same seam the process path uses); without it, compiling a bucket-ingested video can't
open gs://.
        local_src = storage.acquire_local_input(
            source_ref, getattr(cfg, "storage", None)
        )
```

```
        # The ffmpeg stitch is CPU-heavy ? serialize it so a raised drain worker count (for
remote
        # dispatch) doesn't run multiple compiles at once on the box (see _LOCAL_COMPUTE_LANE).
        with _LOCAL_COMPUTE_LANE:
            out_path = compiler.compile_highlights(local_src, time_pairs, out_name)
    except Exception as e:  # noqa: BLE001 ? a stitch/ffmpeg failure is a clean job FAILURE, not
a crash
        logger.error(
            "[compile] stitching failed for video_id=%s: %s", video_id, e, exc_info=True
        )
        return {
            "status": "failed",
            "message": f"Highlight stitching failed: {e}",
            "video_id": video_id,
        }
    finally:
        if local_src:
            storage.cleanup_acquired_local_input(
                source_ref, local_src, getattr(cfg, "storage", None)
            )

    # #463: mirror the reel to the output bucket so /api/highlights can 307 to a signed URL ?
Cloud Run
    # caps a buffered response at ~32 MiB (a real reel is bigger) and the local /tmp copy dies
when the
    # instance recycles. A mirror-upload failure must NOT fail an otherwise-good stitch (the
endpoint
    # still falls back to the local file on the same instance). On the L4 worker the mirror is
how the
    # reel reaches the bucket at all (the worker's local file is discarded when the Job exits).
    reel_uri = reel_object_uri(cfg, video_id, out_name)
    if reel_uri:
        try:
            notify("uploading reel")
            # Thread the active storage settings so the configured backend (S3 endpoint/region,
GCS
            # project/client) is used, not a fresh default.
            storage.put(out_path, reel_uri, config=_storage_settings(cfg))
        except Exception as e:  # noqa: BLE001 ? best-effort mirror; local stitch already
succeeded
            logger.warning("[compile] could not mirror reel to %s: %s", reel_uri, e)

    # `filepath` is server-local (not browser-reachable); also return the download URL so the
SPA needn't
    # hand-build it (P6 / PER_RALLY_SELECTION.md §4.3).
    download_url = f"/api/highlights/{quote(video_id)}/{quote(out_name)}"
    notify("finished")
    return {
        "status": "success",
        "filepath": out_path,
        "download_url": download_url,
        "video_id": video_id,
        "reel_uri": reel_uri or "",
    }


def _dispatch_compile_remote(
    cfg,
    db,
    video_id: str,
    segment_ids: List[int],
    output_filename: str,
    locale: Optional[str],
    notify,
) -> Dict[str, Any]:
```

```
    """#521: run the compile/stitch on the L4 worker instead of the CPU-only control-plane. The
    control-plane owns the DB, so it RESOLVES the reel's segments here (``_resolve_compile``),
stages a
    small compile-spec (resolved time-pairs + the resolved stitching/watermark config + the
source URI)
    to the bucket, dispatches a ``compile`` Cloud Run Job on the L4 (NVENC), bucket-polls the
    done-marker, and returns the same ``{status, filepath, download_url}`` shape the SPA
expects. The
    worker needs NO control-plane DB ? everything it needs rides the spec. Mirrors
    ``_maybe_dispatch_remote`` (the DETECT dispatch)."""
    from backend.compute import CompileJobSpec, get_compute_backend

    notify("resolving")
    try:
        video, kept, out_name = _resolve_compile(
            db, cfg, video_id, segment_ids, output_filename
```

11. assistant (2026-07-08T06:15:29.934Z)

This is the merged code. The swallow-and-return-success logic is confirmed. Now the crux: does the worker `cmd_compile` or the control-plane `_dispatch_compile_remote` verify the reel actually landed in the bucket before declaring success? And do the tests really mock out the real mirror path? Let me read the worker and dispatch code, plus the tests.

12. user (2026-07-08T06:15:31.562Z)

```
1       """Worker dispatcher ? pull from storage ? run the pipeline ? push outputs back.
2
3           python -m backend.worker infer --video-uri <uri> --out-uri <prefix> \
4               [--segmenter wasb_hybrid] [--config config.json] [--set k=v ...] [--scratch DIR]
5           python -m backend.worker train ...        # Gen-0 train-from-bucket (follow-up)
6
7       The URI scheme selects the storage backend via ``backend.storage`` (a bare path /
``local://``
8       is an identity passthrough; ``s3://`` / ``gcs://`` is an object store), so existing
local runs
9       are byte-for-byte unchanged. Combine with ``RALLY_DETECTOR_IMPL=stub`` and
10      ``--set indexing.skip_ai_handoff=true`` for a **secret-free, GPU-free** run (no Gemini
key) ?
11      the path validated end-to-end on DigitalOcean (``docs/SETUP_NEW_MACHINE.md``).
12
13      This is a thin orchestrator: it composes the SAME building blocks the ``backend.main``
CLI uses
14      (validator registry ? segmenter ? report), never a forked pipeline.
15      """
16
17      from __future__ import annotations
18
19      import argparse
20      import csv
21      import json
22      import logging
23      import os
24      import re
25      import sys
26      import tempfile
27      from typing import Any, List
28
29      from backend import storage
30      from backend.compute import ComputeJobSpec, get_compute_backend
31      from backend.config import (
32          StartupCheckError,
33          enforce_startup_checks,
34          load_config,
35          probe_cuda_available,
```

```
    )
    from backend.infrastructure.database import Database
    from backend.pipeline.segmenters.base import segmenter_registry
    from backend.pipeline.validators.base import registry as validator_registry
    from backend.reporting import emit_indexer_report
    from backend.results import Failure
    from backend.utils.ffmpeg import ffprobe_bin  # P2a: single ffmpeg/ffprobe resolver
    from backend.utils.hashing import compute_video_id

    logger = logging.getLogger("backend.worker")


    def _coerce(v: str) -> Any:
        """Coerce a ``--set`` string value to bool/int/float where it round-trips, else
str."""
        low = v.lower()
        if low in ("true", "false"):
            return low == "true"
        for cast in (int, float):
            try:
                return cast(v)
            except ValueError:
                pass
        return v


    def _apply_overrides(config: Any, sets: List[str]) -> None:
        """Apply ``a.b.c=value`` overrides onto the typed config (e.g.
indexing.skip_ai_handoff=true)."""
        for kv in sets or []:
            key, sep, val = kv.partition("=")
            if not sep:
                raise ValueError(f"--set expects k=v, got {kv!r}")
            parts = key.split(".")
            obj = config
            for p in parts[:-1]:
                obj = getattr(obj, p)
            setattr(obj, parts[-1], _coerce(val))


    def _write_intervals_csv(segments: List[dict], path: str) -> None:
        """Decimal-second ``[start,end)`` rallies + shot count / ending reason ? the join-
friendly
        artifact (same schema as the rally-annotator labels)."""
        with open(path, "w", newline="") as f:
            w = csv.writer(f)
            w.writerow(["start", "end", "shot_count", "ending_reason"])
            for s in segments:
                w.writerow(
                    [
                        f"{float(s.get('start_time', 0.0)):.3f}",
                        f"{float(s.get('end_time', 0.0)):.3f}",
                        s.get("shot_count", ""),
                        s.get("ending_reason", s.get("shot_count_confidence", "")),
                    ]
                )


    def _write_report_json(
        video_id: str, segments: List[dict], status: str, path: str, video_uri: str = ""
    ) -> None:
        with open(path, "w") as f:
            json.dump(
                {
                    "video_id": video_id,
```

```
                    "status": status,
                    "segment_count": len(segments),
                    # The SOURCE the detection ran on (CP-rebuild P3, #485): a restart-wiped
                    # control plane re-hydrating from outputs/<md5>/ restores the video row
                    # with its truthful path ? without this, only the segments recover.
                    "video_uri": video_uri,
                    "segments": [
                        {
                            "start": float(s.get("start_time", 0.0)),
                            "end": float(s.get("end_time", 0.0)),
                        }
                        for s in segments
                    ],
                },
                f,
                indent=2,
            )


    def _serialize_and_push_outputs(
        scratch: str,
        out_uri: str,
        video_id: str,
        segments: List[dict],
        status: str,
        storage_cfg: dict,
        video_uri: str = "",
    ) -> List[str]:
        """Write intervals.csv + report.json under ``<scratch>/outputs/<video_id>/`` and
push both to
        ``<out_uri>/outputs/<video_id>/``; return the pushed URIs.

        Shared by the SUCCESS path and the failure path (``_fail``) so a detector
timeout/abort leaves a
        ``status="failed"`` report.json in the bucket ? the control plane's source of truth
        (``orchestration.cached_process_result`` / ``probe_process_cache`` treat any
non-``success``
        report as "no usable result" ? retry, and the restart-recovery poll waits for
report.json to
        EXIST) ? instead of a missing artifact or a false 0-segment ``"success"``."""
        out_local = os.path.join(scratch, "outputs", video_id)
        os.makedirs(out_local, exist_ok=True)
        _write_intervals_csv(segments, os.path.join(out_local, "intervals.csv"))
        _write_report_json(
            video_id,
            segments,
            status,
            os.path.join(out_local, "report.json"),
            video_uri=video_uri,
        )
        out_prefix = out_uri.rstrip("/")
        pushed: List[str] = []
        for fn in sorted(os.listdir(out_local)):
            uri = f"{out_prefix}/outputs/{video_id}/{fn}"
            storage.put(os.path.join(out_local, fn), uri, config=storage_cfg)
            pushed.append(uri)
        return pushed


    def _run_startup_checks(config: Any) -> bool:
        """M5 per-profile startup checks for the worker (the compute process, so it probes
CUDA
        best-effort). Logs warnings; returns False on a FATAL coherence error so the caller
aborts
        cleanly (rc 2) rather than run an expensive job under an incoherent profile. A true
```

```
no-op
157         (returns True, ZERO work) when no deployment.profile is selected.
158
159         The CUDA probe is gated behind an active profile: ``probe_cuda_available()`` lazily
imports
160         torch, and pre-M5 the worker's infer/train path was torch-free (detection is
delegated to a
161         WSL/native subprocess). Probing only when a profile is set keeps the default-OFF
path a true
162         no-op of work, not just of output."""
163         profile = getattr(getattr(config, "deployment", None), "profile", "") or ""
164         cuda = (
165             probe_cuda_available() if profile else None
166         )  # torch-free on the default-OFF path
167         try:
168             enforce_startup_checks(config, cuda_available=cuda, logger=logger)
169             return True
170         except StartupCheckError:
171             return False  # already logged at ERROR by enforce_startup_checks
172
173
174     def _dispatch_remote(compute: Any, args: argparse.Namespace) -> int:
175         """M6: hand ONE infer job to a remote ComputeBackend (Cloud Run) and return its rc
once the
176         outputs are durably in the bucket (the backend bucket-polls the done-marker). The
dispatched
177         container runs this SAME cmd_infer INLINE (compute_target forced local_gpu) ? never
re-dispatch.
178
179         A remote job that never writes a completion marker exits via one of two CLEAN codes
(distinct
180         from the local 2/3), never a raw traceback:
181           * rc 4 ? the bucket poll exhausted its budget (``PolicyTimeout``): the execution
may still be
182             running (the shim best-effort cancels it).
183           * rc 5 ? the execution has TERMINATED without a marker
(``RemoteExecutionFailed``): a worker
184             crash/OOM/pre-marker abort caught FAST by the Executions-API probe, so instead
of the
185             ~65-min opaque wait the operator gets an immediate, log-pointing failure."""
186         from backend.compute import RemoteExecutionFailed
187         from backend.infrastructure.execution_policy import PolicyTimeout
188
189         spec = ComputeJobSpec(
190             video_uri=args.video_uri,
191             out_uri=args.out_uri,
192             segmenter=args.segmenter,
193             config_overrides=tuple(args.set or ()),
194             skip_validation=bool(getattr(args, "skip_validation", False)),
195         )
196         try:
197             result = compute.run_infer(spec)
198         except RemoteExecutionFailed as e:
199             logger.error(
200                 "[worker] remote execution terminated without a completion marker: %s", e
201             )
202             return 5
203         except PolicyTimeout as e:
204             logger.warning(
205                 "[worker] remote compute did not complete (no marker within budget): %s", e
206             )
207             return 4
208         logger.info(
209             "[worker] remote compute done: video_id=%s rc=%d outputs=%s",
210             result.video_id,
```

```
            result.returncode,
            result.outputs_uri,
        )
        return result.returncode


    def _write_done_marker(
        done_uri: str,
        video_id: str,
        returncode: int,
        segment_count: int,
        status: str,
        storage_cfg: dict,
        scratch: str,
    ) -> None:
        """M6: write a tiny JSON completion marker to ``done_uri`` (via the storage layer)
for a remote
        dispatcher's bucket-poll. Best-effort ? never raises into the worker's success
return."""
        try:
            marker = {
                "video_id": video_id,
                "returncode": returncode,
                "segment_count": segment_count,
                "status": status,
            }
            local = os.path.join(scratch, "_done.json")
            with open(local, "w", encoding="utf-8") as f:
                json.dump(marker, f)
            storage.put(local, done_uri, config=storage_cfg)
            logger.info("[worker] wrote completion marker -> %s", done_uri)
        except Exception as e:  # never fail a good run because the marker push hiccuped
            logger.warning("[worker] could not write done-marker %s: %s", done_uri, e)


    def cmd_infer(args: argparse.Namespace) -> int:
        config = load_config(args.config) if args.config else load_config()
        _apply_overrides(config, args.set)
        if not _run_startup_checks(config):
            return 2

        # M6: a cloud compute_target DISPATCHES to a Cloud Run Job instead of running in-
process.
        # DEFAULT-OFF: get_compute_backend returns None for ""/local_gpu/cpu_mock ? fall
through to the
        # unchanged inline path below. The dispatched container runs THIS cmd_infer inline
(the
        # dispatcher forces compute_target=local_gpu), so it never recursively re-
dispatches.
        compute = get_compute_backend(config)
        if compute is not None:
            return _dispatch_remote(compute, args)

        storage_cfg = config.storage.model_dump()

        scratch = args.scratch or tempfile.mkdtemp(prefix="rally-worker-")
        os.makedirs(scratch, exist_ok=True)
        config.output.output_dir = scratch

        # 1. PULL ? local:// / bare path = identity passthrough; s3:// / gcs:// = download
to scratch.
        local_video = storage.fetch(
            args.video_uri, os.path.join(scratch, "in.mp4"), config=storage_cfg
        )
        print(f"[worker] pulled {args.video_uri} -> {local_video}")
```

```
269
270         # 2. IDENTITY ? whole-file MD5 (the labels?video join key; pulled bytes must be
byte-identical).
271         video_id = compute_video_id(local_video)
272
273         # 3. VALIDATE (same registry as the main CLI) ? UNLESS the dispatcher already did.
274         # On the cloud-serving path the CONTROL-PLANE is the authoritative validation gate:
it runs the
275         # SAME validators with ITS resolved config and only dispatches on PASS. Re-
validating here is
276         # redundant, decodes the whole video a SECOND time, and is outright WRONG when the
worker's config
277         # resolves different thresholds than the control-plane's ? observed 2026-07-07: a
5312x2988 clip
278         # (sharpness 11.9) PASSED the control-plane at validation.min_blur_threshold=8.0,
then the worker
279         # REJECTED it at its own default 20.0, so a SUCCESSFUL GPU dispatch failed after the
fact.
280         # --skip-validation (set by the dispatcher in orchestration._maybe_dispatch_remote)
skips it; the
281         # direct ``worker infer`` CLI has no control-plane, so it still validates by
default.
282         if getattr(args, "skip_validation", False):
283             print(
284                 "[worker] validation SKIPPED (--skip-validation): the dispatcher is the
authoritative "
285                 "gate and already validated this input."
286             )
287             val_results, val_context = [], {}
288         else:
289             val_results, val_context = validator_registry.run_all_with_context(
290                 local_video, config
291             )
292         failures = [r for r in val_results if not r.passed]
293         db = Database(os.path.join(scratch, config.output.db_name))
294         db.add_video(
295             video_id,
296             local_video,
297             "failed" if failures else "processed",
298             [r.model_dump() for r in val_results],
299         )
300
301         def _fail(rc: int) -> int:
302             # A worker-side FAILURE (validation, unknown segmenter, or a detector
timeout/abort the
303             # segmenter surfaced as a Failure) must NOT masquerade as a 0-rally "success".
Write + push
304             # a status="failed" report.json/intervals.csv (0 segments) so the control
plane's bucket
305             # truth (submit-cache probe, restart-recovery) and the alpha-user result surface
see an
306             # explicit FAILURE ? the wf294 silent-failure fix. Best-effort: a push hiccup
must never
307             # mask the original failure. THEN the M6 done-marker (real rc + status) so a
remote
308             # dispatcher's bucket-poll terminates FAST instead of hanging until its budget
expires.
309             try:
310                 _serialize_and_push_outputs(
311                     scratch,
312                     args.out_uri,
313                     video_id,
314                     [],
315                     "failed",
316                     storage_cfg,
```

```
                    str(getattr(args, "video_uri", "") or ""),
                )
            except Exception as e:  # never mask the real failure on an artifact-push hiccup
                logger.warning("[worker] could not push failure artifacts: %s", e)
            # DEFAULT-OFF: no --done-uri ? no marker written (the unchanged local run).
            if getattr(args, "done_uri", None):
                _write_done_marker(
                    args.done_uri, video_id, rc, 0, "failed", storage_cfg, scratch
                )
            return rc

        segments: List[dict] = []
        segmenter_actual = None
        segmentation_ran = False
        status = "failed"
        try:
            if failures:
                print(
                    "[worker] validation FAILED: "
                    + "; ".join(f"{r.validator_name}: {r.message}" for r in failures)
                )
                return _fail(2)
            seg_name = args.segmenter or config.indexing.default_segmenter
            segmenter_cls = segmenter_registry.get(seg_name)
            if not segmenter_cls:
                print(
                    f"[worker] unknown segmenter: {seg_name!r} "
                    f"(have {list(segmenter_registry.list_available())})"
                )
                return _fail(2)
            segmenter_actual = seg_name
            result = segmenter_cls(db, config).process_video(
                video_id, local_video, max_frames=args.max_frames
            )
            segmentation_ran = True
            if isinstance(result, Failure):
                print(f"[worker] segmenter FAILED: {result.message}")
                return _fail(3)
            segments = result.value if hasattr(result, "value") else result
            status = "success"
            # Loud, never-silent diagnostic: a 0-segment success is almost always a
misconfig, not an
            # empty match ? the cold-start failure mode (the substrate detector yielded no
usable
            # trajectories). Explain it so a run is analysable from the log alone (re-run -v
for more).
            if not segments:
                detector_impl = (
                    os.environ.get("RALLY_DETECTOR_IMPL")
                    or getattr(config.indexing, "detector_impl", None)
                    or "auto"
                )
                logger.warning(
                    "infer produced 0 segments for video_id=%s (segmenter=%s,
detector_impl=%s). "
                    "The detector substrate yielded no usable trajectories/windows ? check
the detector "
                    "logs above; common causes: a missing ${AI_PROVIDER}_API_KEY with "
                    "skip_ai_handoff=False (the segmenter bails BEFORE detection ? set the
key or "
                    "indexing.skip_ai_handoff=true; cloud-serving now fails this at
startup), native "
                    "runner UNAVAILABLE (weights/repo/WASB_PYTHON env), the WASB env
produced no "
                    "detections, or the input resolution differs from the tuned proxy
```

```
resolution. "
                    "Re-run with -v for the native command + frame counts.",
                    video_id,
                    segmenter_actual,
                    detector_impl,
                )
        finally:
            # Best-effort per-video observability report (never raises) ? parity with the
main CLI.
            emit_indexer_report(
                db=db,
                video_id=video_id,
                video_path=local_video,
                config=config,
                val_results=val_results,
                val_context=val_context,
                segmenter_requested=args.segmenter,
                segmenter_actual=segmenter_actual,
                was_reingest=False,
                segmentation_ran=segmentation_ran,
            )

        # 5. SERIALIZE + 6. PUSH (outputs/<video_id>/?). idempotent on video_id.
        pushed = _serialize_and_push_outputs(
            scratch,
            args.out_uri,
            video_id,
            segments,
            status,
            storage_cfg,
            str(getattr(args, "video_uri", "") or ""),
        )

        print(
            f"[worker] infer done: {len(segments)} segment(s); pushed {len(pushed)}
artifact(s):"
        )
        for u in pushed:
            print("   ", u)

        # M6: a remote dispatcher passes --done-uri; write the SUCCESS completion marker so
its bucket-poll
        # terminates (carries video_id ? the dispatcher's output path + rc=0). Worker
FAILURES (rc 2/3)
        # write their own marker via _fail() above, so the dispatcher fails FAST either way
(no 30-min hang
        # on a bad video). DEFAULT-OFF: no --done-uri ? no marker written (the unchanged
local run).
        if getattr(args, "done_uri", None):
            _write_done_marker(
                args.done_uri, video_id, 0, len(segments), status, storage_cfg, scratch
            )
        return 0


    def _read_compile_spec(spec_uri: str, storage_cfg: dict, scratch: str) -> dict:
        """Fetch + parse the control-plane-staged compile-spec (#521): the resolved
``time_pairs``, the
        resolved ``stitching`` config, the source ``video_uri``, ``video_id``,
``output_name`` + ``locale``.
        A gs://|s3:// URI is downloaded to scratch first (cloud backends require a real
dst)."""
        local = storage.fetch(
            spec_uri, os.path.join(scratch, "compile_spec.json"), config=storage_cfg
        )
```

```
        with open(local, encoding="utf-8") as f:
            data = json.load(f)
        if not isinstance(data, dict):
            raise ValueError(f"compile spec at {spec_uri} is not a JSON object")
        return data


    def _write_compile_marker(
        done_uri: str,
        result: dict,
        video_id: str,
        returncode: int,
        storage_cfg: dict,
        scratch: str,
    ) -> None:
        """#521: write the compile done-marker (rc + status + the reel's
download_url/reel_uri) so the
        dispatcher's bucket-poll terminates and can return the SPA result. Best-effort ?
never raises."""
        try:
            marker = {
                "video_id": video_id,
                "returncode": returncode,
                "status": result.get("status", "failed"),
                "download_url": result.get("download_url", ""),
                "reel_uri": result.get("reel_uri", ""),
                "message": result.get("message", ""),
            }
            local = os.path.join(scratch, "_compile_done.json")
            with open(local, "w", encoding="utf-8") as f:
                json.dump(marker, f)
            storage.put(local, done_uri, config=storage_cfg)
            logger.info("[worker] wrote compile marker -> %s", done_uri)
        except Exception as e:  # never fail a good stitch because the marker push hiccuped
            logger.warning("[worker] could not write compile done-marker %s: %s", done_uri,
e)


    def cmd_compile(args: argparse.Namespace) -> int:
        """#521: run the reel COMPILE/stitch on THIS worker (the L4 has NVENC + 8 vCPU). The
stitch is the
        2026-07-08 CUJ's dominant term (~13 min on the 2-vCPU control-plane for an 11-clip
4K reel). Read
        the control-plane-staged compile-spec (resolved time-pairs + stitching config +
source URI), stitch
        the reel via the SAME ``orchestration._stitch_reel`` core the control-plane uses (so
there is no
        forked stitch pipeline), upload the reel to the bucket, and write a done-marker for
the dispatcher's
        bucket-poll. No control-plane DB is needed ? the spec carries everything the stitch
requires."""
        from backend import orchestration
        from backend.config import StitchingConfig

        config = load_config(args.config) if args.config else load_config()
        _apply_overrides(config, args.set)
        storage_cfg = config.storage.model_dump()

        scratch = args.scratch or tempfile.mkdtemp(prefix="rally-compile-")
        os.makedirs(scratch, exist_ok=True)
        # Clips + the assembled reel land in scratch; the reel is then mirror-uploaded to
the bucket.
        config.output.output_dir = scratch
        # The dispatcher forwards storage.output_root via --set; --out-uri carries the same
value as a
```

```
        # fallback so reel_object_uri (which reads storage.output_root) always resolves the
reel's bucket
        # destination even if the override were dropped.
        if not (config.storage.output_root or "").strip():
            config.storage.output_root = args.out_uri

        spec = _read_compile_spec(args.spec_uri, storage_cfg, scratch)
        video_id = str(spec.get("video_id") or "")
        out_name = str(spec.get("output_name") or "")
        source_ref = str(spec.get("video_uri") or "")
        locale = spec.get("locale")
        time_pairs = [
            (float(pair[0]), float(pair[1])) for pair in (spec.get("time_pairs") or [])
        ]
        stitching = StitchingConfig(**(spec.get("stitching") or {}))

        print(
            f"[worker] compile: video_id={video_id} clips={len(time_pairs)} "
            f"encoder={stitching.encoder} out={out_name}"
        )

        result = orchestration._stitch_reel(
            config,
            stitching,
            video_id,
            source_ref,
            time_pairs,
            out_name,
            locale,
            lambda stage: print(f"[worker] compile: {stage}"),
        )
        status = str(result.get("status") or "failed")
        rc = 0 if status == "success" else 3
        if status != "success":
            print(f"[worker] compile FAILED: {result.get('message')}")

        # DEFAULT-OFF: no --done-uri ? no marker (a direct-CLI stitch). A remote dispatcher
passes it.
        if getattr(args, "done_uri", None):
            _write_compile_marker(args.done_uri, result, video_id, rc, storage_cfg, scratch)
        return rc


    #: Video extensions accepted under <corpus>/videos/ (mirrors the labels/ '.csv' filter
so a
    #: per-video sidecar ? .srt/.json/thumbnail ? can't shadow the real video on a stem
collision).
    _VIDEO_EXTS = (".mp4", ".mov", ".mkv", ".avi", ".webm", ".m4v")

    #: Leading ISO-date prefix on a canonical label name (``labels/<YYYY-MM-
DD>_<name>.rallies.csv``,
    #: docs/DATA_IN_GCS.md §7.1). It disambiguates recycled GoPro names by authoring date
but is NOT
    #: part of the clip's join key, so it's stripped when deriving the videos/ stem.
    _LABEL_DATE_PREFIX = re.compile(r"^\d{4}-\d{2}-\d{2}_")


    def _label_clip_name(fname: str) -> str:
        """Derive the clip name (the ``videos/<name>.<ext>`` join key) from a label
filename.

        Strips both the ``.rallies.csv`` / ``.csv`` suffix and an optional leading ISO-date
prefix, so a
        canonical dated label and a legacy bare label resolve to the SAME bare clip stem:

```

```
        * ``2026-06-22_GX020094.rallies.csv`` -> ``GX020094``       (canonical, date-
stamped)
        * ``GX020094.rallies.csv``            -> ``GX020094``       (legacy bare ? same
stem)
        * ``GX020094_proxy.rallies.csv``      -> ``GX020094_proxy`` (``_proxy`` is part of
the name, kept)
        * ``2026-06-21_mahadevpura_1.rallies.csv`` -> ``mahadevpura_1`` (underscores in
<name> survive)

        Without the date strip, ``--trajectory-source detector`` can't match a dated label
to its bare
        video stem, so every clip is skipped and train aborts with ``<2 videos``
(DATA_IN_GCS §7b #1).
        """
        name = fname.replace(".rallies.csv", "").replace(".csv", "")
        return _LABEL_DATE_PREFIX.sub("", name)


    def _probe_video_fps_width(path: str):
        """Robust ffprobe (fps, frame_width) for a real video. Returns None on ANY failure.

        The detector trajectory is frame-indexed; the harness maps frame->time via fps, so a
WRONG
        fps silently mis-aligns every rally label (e.g. a 60fps clip read as 30 doubles
every time).
        cv2's ``_probe_fps_width`` silently defaults to (30, 1280) on a read miss ? fine for
the
        inference report (it flags the fallback) but corrupting for REAL training data. So
here we
        probe with ffprobe (same tool as ``get_video_duration``) and return None so the
caller SKIPS
        rather than guesses.
        """
        import json
        import subprocess

        try:
            out = subprocess.check_output(
                [
                    ffprobe_bin(),
                    "-v",
                    "error",
                    "-select_streams",
                    "v:0",
                    "-show_entries",
                    "stream=r_frame_rate,width",
                    "-of",
                    "json",
                    path,
                ],
                stderr=subprocess.DEVNULL,
            ).decode()
            st = (json.loads(out).get("streams") or [{}])[0]
            num, _, den = str(st.get("r_frame_rate", "0/1")).partition("/")
            fps = float(num) / float(den or "1")
            width = float(st.get("width") or 0)
            if fps > 0 and width > 0:
                return fps, width
        except (
            subprocess.SubprocessError,
            ValueError,
            KeyError,
            OSError,
            json.JSONDecodeError,
        ):
```

```
            pass
        return None


    def _resolve_train_detector(args: argparse.Namespace, config: Any):
        """Build the trajectory DetectorRunner for `--trajectory-source detector`, honouring
        detector_impl (RALLY_DETECTOR_IMPL / indexing.detector_impl: auto=WSL, native=GPU
box,
        stub=CPU mock). Returns the runner, or prints an error + returns None.

        Reuses the segmenter's `_resolve_runner` seam so the worker and the live CLI build
the
        detector identically (one source of truth). A non-trajectory segmenter (e.g. yolo)
has
        no shuttle detector and is rejected.
        """
        from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter

        seg_cls = segmenter_registry.get(args.segmenter)
        if not seg_cls:
            print(
                f"[worker] unknown segmenter: {args.segmenter!r} "
                f"(have {list(segmenter_registry.list_available())})"
            )
            return None
        seg = seg_cls(None, config)
        if not isinstance(seg, TrajectoryHybridSegmenter):
            print(
                f"[worker] --segmenter {args.segmenter!r} has no shuttle detector; "
                f"use a trajectory hybrid (wasb_hybrid / tracknet_hybrid / fusion_hybrid)."
            )
            return None
        try:
            runner, _ = seg._resolve_runner(config.indexing)
        except NotImplementedError as e:
            # e.g. detector_impl='native' for a family without a native runner (tracknet, or
            # fusion with a tracknet substrate). Fail cleanly (rc!=0), not with a raw
traceback.
            print(f"[worker] detector not available: {e}")
            return None
        ok, msg = runner.healthcheck()
        if not ok:
            print(f"[worker] detector environment not ready: {msg}")
            return None
        print(f"[worker] detector ready ({args.segmenter}): {msg}")
        return runner


    def cmd_train(args: argparse.Namespace) -> int:
        """Gen-0 train-from-bucket: pull labels (+ videos) -> trajectories -> manifest ->
shell out
        to training.gen0.harness (backend must NOT import training) -> push the artifact
bundle.

        Two trajectory sources (the train pipeline + harness are identical either way):
          * ``synth`` (default) ? the CPU "mock GPU": deterministic, label-consistent
synthetic
            shuttle paths (no GPU/WSL), so the whole pipeline is validated on a CPU box /
CI.
          * ``detector`` ? REAL shuttle trajectories: pull each video and run the resolved
            DetectorRunner (native WASB on a GPU box, or stub for a GPU-free wiring test).
This
            is the M3.5 "first real training" path; only the trajectory source flips.
        """
        import subprocess
```

```
650
651         config = load_config(args.config) if args.config else load_config()
652         _apply_overrides(config, args.set)
653         if not _run_startup_checks(config):
654             return 2
655         storage_cfg = config.storage.model_dump()
656
657         scratch = args.scratch or tempfile.mkdtemp(prefix="rally-train-")
658         labels_dir = os.path.join(scratch, "labels")
659         traj_dir = os.path.join(scratch, "trajectories")
660         videos_dir = os.path.join(scratch, "videos")
661         os.makedirs(labels_dir, exist_ok=True)
662         os.makedirs(traj_dir, exist_ok=True)
663
664         corpus = args.corpus_uri.rstrip("/")
665         label_uris = sorted(
666             u
667             for u in storage.list_uris(f"{corpus}/labels/", config=storage_cfg)
668             if u.lower().endswith(".csv")
669         )
670         if len(label_uris) < 2:
671             print(
672                 f"[worker] need >=2 labeled videos for leave-one-video-out; found
{len(label_uris)} "
673                 f"under {corpus}/labels/"
674             )
675             return 2
676
677         # Real-detector source: resolve the runner once + index the bucket's videos/ by
stem.
678         runner = None
679         video_by_stem: dict = {}
680         if args.trajectory_source == "detector":
681             os.makedirs(videos_dir, exist_ok=True)
682             runner = _resolve_train_detector(args, config)
683             if runner is None:
684                 return 2
685             for vuri in storage.list_uris(f"{corpus}/videos/", config=storage_cfg):
686                 vname = storage.parse_uri(vuri).name
687                 if not vname.lower().endswith(_VIDEO_EXTS):
688                     continue  # skip sidecars (.srt/.json/thumbnails) so they can't shadow
the video
689                 video_by_stem[os.path.splitext(vname)[0]] = vuri
690
691         print(f"[worker] trajectory source: {args.trajectory_source}")
692         specs: List[dict] = []
693         for uri in label_uris:
694             fname = storage.parse_uri(uri).name  # e.g. 2026-06-22_GX020094.rallies.csv
695             name = _label_clip_name(fname)  # -> GX020094 (date prefix stripped)
696             local_label = storage.fetch(
697                 uri, os.path.join(labels_dir, fname), config=storage_cfg
698             )
699             if args.trajectory_source == "detector":
700                 vuri = video_by_stem.get(name)
701                 if not vuri:
702                     print(
703                         f"[worker] no video for {name!r} under {corpus}/videos/ ? skipping."
704                     )
705                     continue
706                 local_video = storage.fetch(
707                     vuri,
708                     os.path.join(videos_dir, storage.parse_uri(vuri).name),
709                     config=storage_cfg,
710                 )
711                 video_id = compute_video_id(local_video)
```

```
                # Real fps/width drive the trajectory frame->time mapping; PROBE (don't
guess) ? a
                # wrong fps silently mis-aligns every label, so skip the video if ffprobe
can't read it.
                meta = _probe_video_fps_width(local_video)
                if meta is None:
                    print(
                        f"[worker] could not probe fps/width for {name!r} (ffprobe) ?
skipping (won't guess)."
                    )
                    continue
                fps, frame_width = meta
                traj = runner.run_predict(local_video, traj_dir, video_id=video_id)
                if not traj:
                    print(
                        f"[worker] detector produced no trajectory for {name!r} ? skipping."
                    )
                    continue
                specs.append(
                    {
                        "name": name,
                        "trajectory": traj,
                        "labels": local_label,
                        "fps": fps,
                        "frame_width": frame_width,
                    }
                )
            else:
                from backend.pipeline.detectors.stub_runner import (
                    synth_trajectory_from_labels,
                )

                traj = os.path.join(traj_dir, f"{name}_traj.csv")
                synth_trajectory_from_labels(
                    local_label, traj, fps=args.fps, frame_width=args.frame_width
                )
                specs.append(
                    {
                        "name": name,
                        "trajectory": traj,
                        "labels": local_label,
                        "fps": args.fps,
                        "frame_width": args.frame_width,
                    }
                )

        if len(specs) < 2:
            print(
                f"[worker] need >=2 videos with trajectories for leave-one-video-out; got
{len(specs)}."
            )
            return 2
        manifest_path = os.path.join(scratch, "manifest.json")
        with open(manifest_path, "w", encoding="utf-8") as f:
            json.dump(specs, f, indent=2)
        src_label = (
            "real detector" if args.trajectory_source == "detector" else "CPU-mock stub"
        )
        print(f"[worker] built manifest: {len(specs)} videos ({src_label} trajectories)")

        out_dir = os.path.join(scratch, "artifacts")
        cmd = [
            sys.executable,
            "-m",
            "training.gen0.harness",
```

```
            "--manifest",
            manifest_path,
            "--out",
            out_dir,
            "--version",
            args.version,
        ]
        if args.floor:
            floor_local = (
                storage.fetch(
                    args.floor, os.path.join(scratch, "floor.json"), config=storage_cfg
                )
                if "://" in args.floor
                else args.floor
            )
            cmd += ["--floor", floor_local]
        print("[worker] training:", " ".join(cmd))
        rc = subprocess.run(cmd).returncode
        if rc != 0:
            print(f"[worker] gen0 harness failed (rc={rc})")
            return rc

        # PUSH the versioned artifact bundle (weights.json + manifest.json) to the bucket.
        bundle = os.path.join(out_dir, args.version)
        out_prefix = args.out_uri.rstrip("/")
        pushed = []
        for fn in sorted(os.listdir(bundle)):
            uri = f"{out_prefix}/weights/{args.version}/{fn}"
            storage.put(os.path.join(bundle, fn), uri, config=storage_cfg)
            pushed.append(uri)
        print(f"[worker] train done: pushed {len(pushed)} artifact(s):")
        for u in pushed:
            print("   ", u)
        return 0


    def build_parser() -> argparse.ArgumentParser:
        p = argparse.ArgumentParser(
            prog="python -m backend.worker",
            description="Storage-aware worker: pull ? run pipeline ? push.",
        )
        p.add_argument(
            "-v",
            "--verbose",
            action="store_true",
            help="DEBUG logging (the native detector command, frame counts, per-stage
detail)",
        )
        sub = p.add_subparsers(dest="cmd", required=True)

        pi = sub.add_parser("infer", help="run inference on one video URI and push outputs")
        pi.add_argument(
            "--video-uri", required=True, help="local path / local:// / s3:// / gcs://"
        )
        pi.add_argument(
            "--out-uri",
            required=True,
            help="output prefix (outputs/<video_id>/? is appended)",
        )
        pi.add_argument(
            "--segmenter", default=None, help="default: indexing.default_segmenter"
        )
        pi.add_argument(
            "--config", default=None, help="config.json path (default: ./config.json)"
        )
```

```
        pi.add_argument(
            "--scratch", default=None, help="local scratch dir (default: a temp dir)"
        )
        pi.add_argument("--max-frames", type=int, default=None)
        pi.add_argument(
            "--done-uri",
            default=None,
            help="M6: storage URI for a completion MARKER (video_id + rc) written on a "
            "successful run, so a remote (Cloud Run) dispatcher's bucket-poll terminates. "
            "Default-OFF: omit it for the normal local run.",
        )
        pi.add_argument(
            "--set",
            action="append",
            default=[],
            metavar="k=v",
            help="config override, e.g. --set indexing.skip_ai_handoff=true",
        )
        pi.add_argument(
            "--skip-validation",
            action="store_true",
            help="skip the worker's re-validation: the dispatcher (control-plane) already
ran the "
            "validators with its resolved config and only dispatches on PASS, so re-
validating here is "
            "redundant, decodes the video again, and can contradict the gate. Default-OFF:
the direct "
            "`worker infer` CLI still validates.",
        )
        pi.set_defaults(func=cmd_infer)

        pc = sub.add_parser(
            "compile",
            help="#521: stitch a highlight reel on this worker (L4 NVENC) from a staged
compile-spec",
        )
        pc.add_argument(
            "--spec-uri",
            required=True,
            help="storage URI of the control-plane-staged compile-spec JSON (time-pairs +
stitching "
            "config + source uri + video_id)",
        )
        pc.add_argument(
            "--out-uri",
            default="",
            help="output ROOT (gs://? bucket); the reel is mirrored to
highlights/<video_id>/<name>. "
            "Also forwarded via --set storage.output_root; either resolves the reel path.",
        )
        pc.add_argument(
            "--done-uri",
            default=None,
            help="storage URI for the compile completion MARKER (rc + download_url), so a
remote "
            "dispatcher's bucket-poll terminates. Default-OFF: omit for a direct-CLI
stitch.",
        )
        pc.add_argument(
            "--config", default=None, help="config.json path (default: ./config.json)"
        )
        pc.add_argument(
            "--scratch", default=None, help="local scratch dir (default: a temp dir)"
        )
        pc.add_argument(
```

```
            "--set",
            action="append",
            default=[],
            metavar="k=v",
            help="config override, e.g. --set storage.output_root=gs://bucket",
        )
        pc.set_defaults(func=cmd_compile)

        pt = sub.add_parser(
            "train",
            help="Gen-0 train-from-bucket (labels [+videos] -> trajectories -> harness)",
        )
        pt.add_argument(
            "--corpus-uri",
            required=True,
            help="prefix containing labels/ (+ videos/ for "
            "--trajectory-source detector), e.g. s3://bucket or gcs://bucket",
        )
        pt.add_argument(
            "--out-uri",
            required=True,
            help="output prefix (weights/<version>/ is appended)",
        )
        pt.add_argument("--version", required=True, help="semver for the artifact bundle")
        pt.add_argument(
            "--trajectory-source",
            choices=["synth", "detector"],
            default="synth",
            help="synth = CPU-mock label-consistent trajectories (default, no GPU); "
            "detector = REAL shuttle trajectories via the resolved DetectorRunner "
            "(needs videos/ in the corpus; honours detector_impl).",
        )
        pt.add_argument(
            "--segmenter",
            default="wasb_hybrid",
            help="trajectory hybrid whose detector is used for --trajectory-source
detector",
        )
        pt.add_argument(
            "--floor", default=None, help="floor baseline JSON (local path or storage URI)"
        )
        pt.add_argument("--config", default=None)
        pt.add_argument("--scratch", default=None)
        pt.add_argument(
            "--fps",
            type=float,
            default=30.0,
            help="synth trajectory fps (detector probes the video)",
        )
        pt.add_argument(
            "--frame-width",
            type=float,
            default=1280.0,
            help="synth trajectory frame width (detector probes the video)",
        )
        pt.add_argument("--set", action="append", default=[], metavar="k=v")
        pt.set_defaults(func=cmd_train)
        return p


    def _setup_logging(verbose: bool) -> None:
        """Configure logging for the worker CLI so the pipeline's existing INFO/ERROR
diagnostics are
        VISIBLE ? the worker previously ran with no logging config, so every
segmenter/detector log
```

```
956         (detector choice, 'Running native WASB inference', availability failures) was
silently dropped.
957         That is exactly why a 0-rally cold-start gave nothing to analyse. Configured here
(the CLI
958         entry), not at import, so importing the module never reconfigures a caller's
logging."""
959         # force=True: basicConfig is a no-op if the root already has handlers (e.g. a
library
960         # configured logging at import), which would leave the level at WARNING and re-
suppress the
961         # diagnostics. Forcing reconfiguration means the worker reliably controls its own
verbosity.
962         logging.basicConfig(
963             level=logging.DEBUG if verbose else logging.INFO,
964             format="%(asctime)s [%(levelname)s] %(name)s: %(message)s",
965             datefmt="%H:%M:%S",
966             force=True,
967         )
968
969
970     def main(argv=None) -> int:
971         args = build_parser().parse_args(argv)
972         _setup_logging(getattr(args, "verbose", False))
973         return args.func(args)
974
975
976     if __name__ == "__main__":
977         sys.exit(main())
978
```

13. assistant (2026-07-08T06:15:53.478Z)

Now I've confirmed the worker side: `cmd_compile` (worker/__main__.py:513-514) sets `rc = 0 if status == "success"` purely from `_stitch_reel`'s return, with no post-mirror verification. Let me read the rest of `_dispatch_compile_remote` (control-plane side ? does IT verify the reel exists?), and check the test coverage.

14. user (2026-07-08T06:15:56.281Z)

```
    """#521: run the compile/stitch on the L4 worker instead of the CPU-only control-plane. The
    control-plane owns the DB, so it RESOLVES the reel's segments here (``_resolve_compile``),
stages a
    small compile-spec (resolved time-pairs + the resolved stitching/watermark config + the
source URI)
    to the bucket, dispatches a ``compile`` Cloud Run Job on the L4 (NVENC), bucket-polls the
    done-marker, and returns the same ``{status, filepath, download_url}`` shape the SPA
expects. The
    worker needs NO control-plane DB ? everything it needs rides the spec. Mirrors
    ``_maybe_dispatch_remote`` (the DETECT dispatch)."""
    from backend.compute import CompileJobSpec, get_compute_backend

    notify("resolving")
    try:
        video, kept, out_name = _resolve_compile(
            db, cfg, video_id, segment_ids, output_filename
        )
    except _CompileError as e:
        return {"status": "failed", "message": e.detail, "video_id": video_id}

    stitching = getattr(cfg, "stitching", None) or StitchingConfig()
    storage_cfg = getattr(cfg, "storage", None)
    out_root = (getattr(storage_cfg, "output_root", "") or "").strip()
    compute = get_compute_backend(cfg, target_override="cloud_run") if out_root else None
    if compute is None:
        # Compile was placed on the L4 but the cloud dispatch isn't actually wired here (no
```

```
output
        # bucket / Cloud Run env). Fall back to the in-process stitch rather than fail ? a slow
reel
        # beats no reel. (The placement guard means this is only reachable in a mis-provisioned
setup.)
        logger.warning(
            "[compile] compile placed on the L4 but no cloud backend available (out_root=%r) ? "
            "stitching in-process on the control-plane instead.",
            out_root,
        )
        time_pairs = [(s["start_time"], s["end_time"]) for s in kept]
        return _stitch_reel(
            cfg, stitching, video_id, video["filepath"], time_pairs, out_name, locale, notify
        )

    # Build the compile-spec the worker consumes. time-pairs are JSON lists (frozen-safe on the
wire);
    # the stitching config is the control-plane's RESOLVED one, with the encoder swapped to the
L4's
    # (deployment.cloud_stitch_encoder, NVENC by default) so the worker hardware-encodes the
reel.
    stitch_dump = stitching.model_dump()
    stitch_dump["encoder"] = getattr(
        getattr(cfg, "deployment", None), "cloud_stitch_encoder", "h264_nvenc"
    )
    spec_payload = {
        "video_id": video_id,
        "video_uri": video["filepath"],
        "output_name": out_name,
        "locale": locale,
        "time_pairs": [
            [float(s["start_time"]), float(s["end_time"])] for s in kept
        ],
        "stitching": stitch_dump,
    }
    token = uuid.uuid4().hex
    spec_uri = f"{out_root.rstrip('/')}/_compile/{token}/spec.json"
    try:
        notify("staging compile spec")
        with tempfile.TemporaryDirectory(prefix="cr-compile-") as td:
            local_spec = os.path.join(td, "spec.json")
            with open(local_spec, "w", encoding="utf-8") as f:
                json.dump(spec_payload, f)
            storage.put(local_spec, spec_uri, config=_storage_settings(cfg))
    except Exception as e:  # noqa: BLE001 ? a staging failure is a clean job failure, not a
crash
        logger.error(
            "[compile] failed to stage the compile spec to %s: %s", spec_uri, e, exc_info=True
        )
        return {
            "status": "failed",
            "message": f"cloud compile: could not stage the compile spec: {e}",
            "video_id": video_id,
        }

    # Forward the bucket coordinates so the worker (no DEPLOYMENT_PROFILE) can read the spec,
mirror the
    # reel, and write the marker ? mirrors how _maybe_dispatch_remote forwards the WASB knobs
for detect.
    overrides = [f"storage.output_root={out_root}"]
    backend_name = (getattr(storage_cfg, "backend", "") or "").strip()
    if backend_name:
        overrides.append(f"storage.backend={backend_name}")
    spec = CompileJobSpec(
        out_uri=out_root,
```

```
        spec_uri=spec_uri,
        video_id=video_id,
        output_name=out_name,
        config_overrides=tuple(overrides),
    )

    notify("dispatching stitch (cloud_run)")
    logger.info(
        "[compile] dispatching stitch for video_id=%s -> Cloud Run Job (out=%s, %d clip(s))",
        video_id,
        out_root,
        len(kept),
    )
    try:
        result = compute.run_compile(
            spec, on_wait=lambda elapsed: notify(_compile_wait_message(elapsed))
        )
    except Exception as e:  # noqa: BLE001 ? any dispatch/poll error is a clean job failure, not
a 500
        logger.error("[compile] cloud compile dispatch error: %s", e, exc_info=True)
        return {
            "status": "failed",
            "message": f"cloud compile dispatch error: {e}",
            "video_id": video_id,
        }
    if result.returncode != 0:
        msg = str(
            result.report.get("message")
            or f"cloud compile job failed (returncode {result.returncode})."
        )
        return {"status": "failed", "message": msg, "video_id": video_id}

    download_url = str(
        result.report.get("download_url")
        or f"/api/highlights/{quote(video_id)}/{quote(out_name)}"
    )
    logger.info(
        "[COMPILE] cloud_run stitched: execution=%s out=%s video_id=%s",
        result.execution,
        result.outputs_uri,
        video_id,
    )
    notify("finished")
    return {
        "status": "success",
        "filepath": str(result.report.get("reel_uri") or result.outputs_uri or ""),
        "download_url": download_url,
        "video_id": video_id,
        "compute": {
            "path": "cloud_run",
            "execution": result.execution,
            "job": os.environ.get("CLOUD_RUN_JOB", "rally-worker"),
            "region": os.environ.get("CLOUD_RUN_REGION", "asia-south1"),
        },
    }
```

15. user (2026-07-08T06:15:57.206Z)

```
952:def test_parser_compile_accepts_spec_and_done_uris():
989:def test_cmd_compile_stitches_and_writes_marker(tmp_path, monkeypatch):
990:    """cmd_compile reads the staged spec, runs the SHARED _stitch_reel core (VideoCompiler
stubbed so
991:    no ffmpeg), mirrors the reel, and writes a compile done-marker for the dispatcher's
bucket-poll."""
1012:            "reel_uri": f"gs://b/highlights/{video_id}/{out_name}",
```

```
1015:    monkeypatch.setattr(orch, "_stitch_reel", _fake_stitch)
1048:def test_cmd_compile_failure_returns_rc3_and_failed_marker(tmp_path, monkeypatch):
1062:        "_stitch_reel",
1089:def test_cmd_compile_no_done_uri_writes_no_marker(tmp_path, monkeypatch):
1100:        orch, "_stitch_reel", lambda *a, **k: {"status": "success", "video_id": "vidReel"}
```

16. assistant (2026-07-08T06:16:11.540Z)

Now let me verify the full test-coverage picture: search ALL test files for any test that exercises the real `_stitch_reel` (not mocked) and specifically a mirror/`storage.put` failure, plus check `test_compile_dispatch.py` / `test_compute_backend.py` for the fake client.

17. user (2026-07-08T06:16:13.407Z)

```
tests\test_compile_dispatch.py:174:    """DEFAULT (compile on the control-plane):
_execute_compile stitches in-process via _stitch_reel
tests\test_compile_dispatch.py:188:    monkeypatch.setattr(orch, "_stitch_reel", _fake_stitch)
tests\test_worker.py:990:    """cmd_compile reads the staged spec, runs the SHARED _stitch_reel
core (VideoCompiler stubbed so
tests\test_worker.py:1015:    monkeypatch.setattr(orch, "_stitch_reel", _fake_stitch)
tests\test_worker.py:1062:        "_stitch_reel",
tests\test_worker.py:1100:        orch, "_stitch_reel", lambda *a, **k: {"status": "success",
"video_id": "vidReel"}
```

18. user (2026-07-08T06:16:13.787Z)

```
985             json.dump(spec, f)
986         return spec_uri
987
988
989     def test_cmd_compile_stitches_and_writes_marker(tmp_path, monkeypatch):
990         """cmd_compile reads the staged spec, runs the SHARED _stitch_reel core
(VideoCompiler stubbed so
991         no ffmpeg), mirrors the reel, and writes a compile done-marker for the dispatcher's
bucket-poll."""
992         import backend.orchestration as orch
993
994         bucket = tmp_path / "bucket"
995         bucket.mkdir()
996         source = tmp_path / "src.mp4"
997         source.write_bytes(b"SRC")
998         spec_uri = _write_compile_spec(bucket, "vidReel", source, "reel.mp4")
999         done = str(bucket / "_compile" / "tok.done")
1000
1001        seen = {}
1002
1003        def _fake_stitch(cfg, stitching, video_id, source_ref, time_pairs, out_name, locale,
notify):
1004            seen["encoder"] = stitching.encoder
1005            seen["pairs"] = time_pairs
1006            seen["source"] = source_ref
1007            notify("stitching")
1008            return {
1009                "status": "success",
1010                "video_id": video_id,
1011                "download_url": f"/api/highlights/{video_id}/{out_name}",
1012                "reel_uri": f"gs://b/highlights/{video_id}/{out_name}",
1013            }
1014
1015        monkeypatch.setattr(orch, "_stitch_reel", _fake_stitch)
1016
1017        cfg = tmp_path / "c.json"
1018        cfg.write_text('{"storage": {"backend": "local"}}')
1019        rc = wmain.cmd_compile(
```

```
            wmain.build_parser().parse_args(
                [
                    "compile",
                    "--spec-uri",
                    spec_uri,
                    "--out-uri",
                    str(bucket),
                    "--done-uri",
                    done,
                    "--config",
                    str(cfg),
                    "--scratch",
                    str(tmp_path / "s"),
                    "--set",
                    f"storage.output_root={bucket}",
                ]
            )
        )
        assert rc == 0
        # the shared stitch core got the resolved pairs + the NVENC encoder from the spec
        assert seen["encoder"] == "h264_nvenc"
        assert seen["pairs"] == [(2.0, 6.0), (10.0, 14.0)]
        assert seen["source"] == str(source)
        marker = json.loads(open(done, encoding="utf-8").read())
        assert marker["returncode"] == 0 and marker["status"] == "success"
        assert marker["download_url"] == "/api/highlights/vidReel/reel.mp4"


    def test_cmd_compile_failure_returns_rc3_and_failed_marker(tmp_path, monkeypatch):
        """A stitch failure surfaces as rc 3 + a failed marker (rc != 0) so the dispatcher's
poll
        terminates and the SPA sees a clean failure."""
        import backend.orchestration as orch

        bucket = tmp_path / "bucket"
        bucket.mkdir()
        source = tmp_path / "src.mp4"
        source.write_bytes(b"SRC")
        spec_uri = _write_compile_spec(bucket, "vidReel", source, "reel.mp4")
        done = str(bucket / "_compile" / "tok.done")

        monkeypatch.setattr(
            orch,
            "_stitch_reel",
            lambda *a, **k: {"status": "failed", "message": "ffmpeg blew up", "video_id":
"vidReel"},
        )
        cfg = tmp_path / "c.json"
        cfg.write_text('{"storage": {"backend": "local"}}')
        rc = wmain.cmd_compile(
            wmain.build_parser().parse_args(
                [
                    "compile",
                    "--spec-uri",
                    spec_uri,
                    "--out-uri",
                    str(bucket),
                    "--done-uri",
                    done,
                    "--config",
                    str(cfg),
                    "--scratch",
                    str(tmp_path / "s"),
                ]
            )
```

```
1083        )
1084        assert rc == 3
1085        marker = json.loads(open(done, encoding="utf-8").read())
1086        assert marker["returncode"] == 3 and marker["status"] == "failed"
1087
1088
1089    def test_cmd_compile_no_done_uri_writes_no_marker(tmp_path, monkeypatch):
1090        """DEFAULT-OFF: a direct-CLI stitch with no --done-uri writes no marker."""
1091        import backend.orchestration as orch
1092
1093        bucket = tmp_path / "bucket"
1094        bucket.mkdir()
1095        source = tmp_path / "src.mp4"
1096        source.write_bytes(b"SRC")
1097        spec_uri = _write_compile_spec(bucket, "vidReel", source, "reel.mp4")
1098
1099        monkeypatch.setattr(
```

19. assistant (2026-07-08T06:17:01.575Z)

I've confirmed all 4 `_stitch_reel` references in tests are monkeypatches. But a test could invoke the real `_stitch_reel` transitively via `cmd_compile` without mentioning it by name. Let me verify `test_compute_backend.py` and `test_compile_dispatch.py` don't drive the real worker mirror path with an injected `storage.put` failure.

20. user (2026-07-08T06:17:05.897Z)

```
160         by the control-plane first."""
161         db, cfg, video_id, seg_ids, _bucket, monkeypatch = compile_env
162         fake = _FakeCompileClient()
163         _inject_fake(monkeypatch, fake)
164
165         out = orch._execute_compile(
166             cfg, db, _job(video_id, seg_ids, "myreel-sd.mp4", rendition="sd")
167         )
168
169         assert out["status"] == "success"
170         assert out["filename"] == "myreel-sd.mp4"
171         assert out["rendition"] == "sd"
172         assert fake.spec is not None
173         assert fake.spec["output_name"] == "myreel-sd.mp4"
174         assert fake.spec["rendition"] == "sd"
175         assert fake.spec["stitching"]["renditions"]["sd"]["max_width"] == 854
176         assert fake.spec["stitching"]["renditions"]["sd"]["max_height"] == 480
177         assert len(fake.calls) == 1
178
179
180     def test_compile_dispatch_forwards_stitch_config_overrides(compile_env):
181         """The worker Job has no DEPLOYMENT_PROFILE, so the dispatch forwards the bucket
coordinates
182         (storage.output_root/backend) ? else the worker can't read the spec or mirror the
reel."""
183         db, cfg, video_id, seg_ids, bucket, monkeypatch = compile_env
184         fake = _FakeCompileClient()
185         _inject_fake(monkeypatch, fake)
186         orch._execute_compile(cfg, db, _job(video_id, seg_ids))
187         joined = " ".join(fake.calls[0])
188         assert f"storage.output_root={bucket}" in joined
189         assert "storage.backend=local" in joined
190
191
192     def test_compile_cloud_failure_returns_failed(compile_env):
193         """A worker stitch failure (rc != 0) surfaces as a failed job with the worker's
message ? the SPA
194         gets a clean verdict, not a crash."""
```

```
195         db, cfg, video_id, seg_ids, _bucket, monkeypatch = compile_env
196         fake = _FakeCompileClient(returncode=3)
197         _inject_fake(monkeypatch, fake)
198 
199         out = orch._execute_compile(cfg, db, _job(video_id, seg_ids))
200         assert out["status"] == "failed"
201         assert "stitch blew up" in out["message"]
202         assert out["video_id"] == video_id
203 
204 
205     def test_compile_default_stitches_in_process_no_dispatch(compile_env, monkeypatch):
206         """DEFAULT (compile on the control-plane): _execute_compile stitches in-process via
_stitch_reel
207         and NEVER dispatches to Cloud Run. The stitch core is stubbed so no real ffmpeg
runs."""
208         db, cfg, video_id, seg_ids, _bucket, _mp = compile_env
209         # flip compile back to the control-plane (the default) ? everything else stays
cloud-wired
210         cfg.deployment.phase_placement.compile = "control_plane"
211         fake = _FakeCompileClient()
212         _inject_fake(monkeypatch, fake)
213         seen = {}
214 
215         def _fake_stitch(c, stitching, vid, source, pairs, name, locale, notify,
rendition="original"):
216             seen["pairs"] = pairs
217             seen["source"] = source
218             seen["rendition"] = rendition
219             return {"status": "success", "video_id": vid, "download_url":
f"/api/highlights/{vid}/{name}"}
220 
221         monkeypatch.setattr(orch, "_stitch_reel", _fake_stitch)
222 
223         out = orch._execute_compile(cfg, db, _job(video_id, seg_ids, "local.mp4"))
224         assert out["status"] == "success"
225         assert fake.calls == []  # NEVER dispatched ? the control-plane stitched in-process
226         assert seen["pairs"] == [(2.0, 6.0), (10.0, 14.0), (20.0, 22.0)]
227         assert seen["source"].endswith("source.mp4")
228         assert seen["rendition"] == "original"
229 
```

21. user (2026-07-08T06:17:06.378Z)

```
=== test_compute_backend: compile/CloudRunJobClient refs ===
3:All GPU-free + cloud-free: the Cloud Run trigger is a FAKE ``CloudRunJobClient`` that
simulates the
4:worker (writes the done-marker + report to a local "bucket" = a tmp dir), so the
dispatch?bucket-poll
20:from backend.compute.cloud_run import CloudRunCompute, CloudRunJobClient
24:# A fast, no-wait policy (the fake client writes the marker synchronously, so the first poll
hits).
28:class _FakeRunClient(CloudRunJobClient):
29:    """Simulates a Cloud Run Job: on dispatch, write the report + done-marker into the local
bucket
76:class _NoopRunClient(CloudRunJobClient):
94:class _NeverDoneClient(CloudRunJobClient):
117:class _CrashedNoMarkerClient(CloudRunJobClient):
176:    """The real GoogleCloudRunJobClient must fail loudly when GOOGLE_CLOUD_PROJECT is unset
?
180:    from backend.compute.cloud_run import GoogleCloudRunJobClient
182:    client = GoogleCloudRunJobClient()
195:    from backend.compute.cloud_run import GoogleCloudRunJobClient
197:    client = GoogleCloudRunJobClient()
598:    from backend.compute.cloud_run import GoogleCloudRunJobClient
600:    client = GoogleCloudRunJobClient()
```

```
764:def test_worker_writes_done_marker_on_success(tmp_path, monkeypatch):
765:    """The container side: cmd_infer with --done-uri writes a completion marker (video_id +
rc) so a
834:    """DEFAULT-OFF: without --done-uri the inline run writes NO marker (today's
behaviour)."""
888:def test_worker_writes_failure_marker_on_validation_fail(tmp_path, monkeypatch):
889:    """The container side: a worker-side FAILURE (validation rc=2) with --done-uri writes a
marker
948:def test_worker_failure_writes_no_marker_without_done_uri(tmp_path, monkeypatch):
949:    """DEFAULT-OFF: a worker failure with NO --done-uri writes no marker (the unchanged
local run)."""
1022:    # The cloud worker reads the gs:// input + writes the gs:// outputs/done-marker through
1084:    client = _FakeRunClient()  # still writes report.json so the post-marker read succeeds
1108:class _FakeCompileClient(CloudRunJobClient):
1109:    """Simulates the Cloud Run COMPILE worker: on dispatch, write a reel done-marker into
the local
1110:    bucket exactly where the real ``cmd_compile`` would, so ``run_compile``'s bucket-poll
resolves."""
1137:def test_run_compile_dispatches_and_returns_marker(tmp_path):
1138:    """run_compile dispatches a `compile` subcommand (spec + done marker), bucket-polls,
and returns
1147:    result = backend.run_compile(
1168:def test_run_compile_surfaces_nonzero_returncode(tmp_path):
1178:    result = backend.run_compile(
1184:def test_compile_marker_prefix_distinct_from_infer(tmp_path):
1194:    backend.run_compile(
1201:def test_run_compile_default_backend_raises_not_implemented():
1203:    CloudRunCompute overrides run_compile) ? never a silent no-op."""
1211:        _InProc().run_compile(
```

22. assistant (2026-07-08T06:17:51.331Z)

I have now fully confirmed both halves of the claim. Let me consolidate the findings.

Failure mode is real (confirmed in code):
- `backend/orchestration.py` `_stitch_reel` (HEAD): after a successful local stitch, `storage.put(out_path, reel_uri, ...)` is wrapped in `except Exception ... logger.warning("[compile] could not mirror reel to %s: %s", ...)` ? the mirror failure is swallowed, then the function unconditionally returns `{"status": "success", ...}`. The comment right above it even admits: *"On the L4 worker the mirror is how the reel reaches the bucket at all (the worker's local file is discarded when the Job exits)"* ? yet still swallows. The CP-path justification (*"the endpoint still falls back to the local file on the same instance"*) does not hold on the worker, so the swallow is unguarded silent data loss there.
- `reel_uri` is non-empty on the worker: `cmd_compile` sets `config.storage.output_root` from `--set`/`--out-uri` (worker/__main__.py:485-486), and `reel_object_uri` reads `output_root`. So the mirror IS attempted and its failure IS reachable/swallowed.
- No verification anywhere downstream: `cmd_compile` (worker/__main__.py:513-514) sets `rc = 0 if status == "success" else 3` purely from the return value; `_write_compile_marker` copies the status through; and control-plane `_dispatch_compile_remote` returns `{"status":"success", download_url, ...}` whenever `result.returncode == 0`, with no `storage.exists(reel_uri)` check. So a swallowed worker-side mirror failure propagates as a SUCCESS with a `download_url` the SPA later polls, but the reel is neither in the bucket nor recoverable from the exited worker.

Coverage gap is real (confirmed): The real `_stitch_reel` (with its `storage.put` mirror + swallow) is never executed by any test. All references mock it: `tests/test_worker.py:1015,1062,1100` (all three `cmd_compile` tests) and `tests/test_compile_dispatch.py:221` (CP in-process path). The dispatch/compute tests use `_FakeCompileClient` (`tests/test_compile_dispatch.py:1108`) which *"write[s] a reel done-marker into the local bucket exactly where the real cmd_compile would"* ? bypassing the real worker entirely. No test injects a `storage.put` mirror failure to assert the resulting status/marker, so the false-success-on-lost-reel path is entirely uncovered.

I could not refute this: both the failure mode and the absence of any covering test are concretely present in the merged code.

23. user (2026-07-08T06:18:04.580Z)

```
Structured output provided successfully
```